

CSD_03 — Artificial Intelligence and Machine Learning

Paper-II, Units 6 and 7 — *degree-only*. Official syllabus wording:

Unit 6 — AI: "Knowledge and Reasoning: Building a Knowledge Base. Learning from Observations: Inductive Learning, Explanation Based Learning. Pattern Recognition: Structured Description, Symbolic Description, Object Identification, Speech Recognition. Programming Language: LISP, PROLOG."

Unit 7 — Machine Learning: "Knowledge-level vs symbol-level learning. Supervised Learning: Distance-based methods, Nearest-Neighbours. Unsupervised Learning Clustering: K-means/Kernel K-means.

Statistical Learning Theory: Ensemble Methods (Boosting, Bagging, Random Forests)."

This file covers both. Nothing in the CS (Diploma) or Forensic syllabi overlaps — every hour here pays into exactly one paper.

Why this matters / where it appears

AI and ML are the two Paper-II units that **only ever appear in the 10- and 15-mark sections**, never in the cheap 5-markers. From the 2025 Paper-II:

- **Q14 (10 marks): knowledge-level vs symbol-level learning** — a direct §7.1 question.
- **Q15 (10 marks): k-Nearest Neighbours; factors influencing its performance** — §7.3.
- **Q19 (15 marks): building a knowledge base in AI; key challenges** — §6.2.
- **MCQ Q90–100 (11% of the MCQ paper): AI and machine learning.**

So this unit is worth roughly **35 descriptive marks and 22 MCQ marks** — do not treat it as optional. The good news: the syllabus names *specific* topics, so preparation is well-targeted. The examinable skill is the same as elsewhere — **work the numericals by hand** (A*, alpha-beta, Bayes, ID3 entropy, k-NN, K-means, confusion matrix). A 15-mark ML answer is a filled-in table with a computed answer, not prose.

Tick-box checklist

§	Topic	Depth	Done
1	Intelligent agents; agent types; environment properties	5-mark	☐
2	Uninformed search: BFS/DFS/UCS/DLS/IDDFS + comparison table	10-mark	☐
3	Informed search: greedy, A* worked , admissibility, heuristics	15-mark	☐
4	Adversarial search: minimax + alpha-beta worked	15-mark	☐
5	Constraint satisfaction problems	5-mark	☐
6	Knowledge & reasoning; building a knowledge base ; representation schemes	15-mark, asked 2025	☐
7	Logic: propositional & predicate; unification, resolution/refutation worked ; chaining	15-mark	☐
8	Uncertainty: Bayes worked , Bayesian networks, fuzzy logic	10-mark	☐
9	Learning: ID3 decision tree worked , inductive, explanation-based	15-mark	☐

10	Pattern recognition; expert systems	10-mark	☐
11	LISP and PROLOG — syntax + example programs	10-mark, MCQ	☐
12	ML basics; knowledge- vs symbol-level ; supervised/unsupervised/reinforcement	10-mark, asked 2025	☐
13	k-NN worked ; distance metrics; curse of dimensionality	10-mark, asked 2025	☐
14	K-means worked iteration ; kernel K-means; hierarchical	15-mark	☐
15	Ensembles (bagging, AdaBoost, random forests); bias-variance	10-mark	☐
16	Overfitting, cross-validation, confusion matrix + precision/recall/F1 worked	10-mark	☐

PART A — ARTIFICIAL INTELLIGENCE (Unit 6)

1. Intelligent agents and environments

Concept — in plain English first

AI is often framed as building **rational agents**: an **agent** is anything that **perceives** its environment through **sensors** and **acts** on it through **actuators**. A **rational agent** chooses the action that maximises its expected performance measure, given what it has perceived and what it knows. That framing — perceive, decide, act — organises the whole subject.

Key points

Agent = architecture + program. The agent program maps the **percept sequence** to an **action**.

The five agent types (in increasing sophistication):

Agent type	How it decides	Note
Simple reflex	Condition–action rules on the <i>current</i> percept only	No memory; fails if the world is partially observable
Model-based reflex	Keeps an internal state (model of the world)	Handles partial observability
Goal-based	Chooses actions that move toward a goal	Needs search/planning
Utility-based	Maximises a utility function (degree of happiness), not just goal/no-goal	Trades off conflicting goals
Learning	Improves with experience via a learning element + critic	Can start knowing little

PEAS — how to specify a task environment: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors. (E.g. a taxi driver: P = safe, fast, legal; E = roads, traffic; A = steering, brake; S = cameras, GPS.)

Environment properties (learn these pairs — MCQ favourites):

Property	Two ends
Observability	Fully vs partially observable
Agents	Single vs multi-agent

Predictability	Deterministic vs stochastic
Episodes	Episodic vs sequential
Change	Static vs dynamic
Values	Discrete vs continuous
Knowledge	Known vs unknown

Chess: fully observable, multi-agent, deterministic, sequential, (semi-)static, discrete. Taxi driving: partially observable, multi-agent, stochastic, sequential, dynamic, continuous.

Likely exam questions

1. What is an intelligent agent? Explain agent structure (sensors, actuators, agent program). **[5]**
2. Explain the types of agents with examples. **[10]**
3. What is PEAS? Give the PEAS description of one task environment. **[5]**
4. Explain the properties used to characterise task environments. **[10]**

MCQ traps

- A **simple reflex** agent has **no internal state**; a **model-based** one does.
- **Rational** $\neq$ **omniscient** — rationality is about expected performance given percepts, not perfect outcomes.
- Chess is **deterministic**; backgammon is **stochastic** (dice).

2. Uninformed (blind) search

Concept — in plain English first

Search solves problems by exploring a **state space**: an initial state, a set of actions, a goal test, and a path cost. **Uninformed** search has *no* information about how close a state is to the goal — it explores blindly, in a fixed order. It differs from informed search only in that it cannot prefer a promising direction.

A search problem = {**initial state, actions, transition model, goal test, path cost**}. The search builds a tree; we measure it by four criteria: **completeness** (finds a solution if one exists), **optimality** (finds the least-cost one), **time**, **space**.

Key points — the algorithms

Let **b** = branching factor, **d** = depth of shallowest goal, **m** = maximum depth.

Algorithm	Strategy	Complete?	Optimal?	Time	Space
BFS (Breadth-First)	Expand shallowest node; queue (FIFO)	Yes (finite b)	Yes* (unit cost)	$O(b^d)$	$O(b^d)$ (memory-heavy)
UCS (Uniform-Cost)	Expand lowest path cost $g(n)$; priority queue	Yes	Yes (any positive cost)	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$	high
DFS (Depth-First)	Expand deepest node; stack (LIFO)	No (infinite/loops)	No	$O(b^m)$	$O(bm)$ (memory-light)

DLS (Depth-Limited)	DFS to depth limit L	If $L \geq d$	No	$O(b^L)$	$O(bL)$
IDDFS (Iterative Deepening)	DLS with increasing $L = 0, 1, 2, \dots$	Yes	Yes* (unit cost)	$O(b^d)$	$O(bd)$

* optimal only when all step costs are equal.

The key trade-off: BFS is complete and optimal but needs exponential *memory*; DFS is memory-cheap but neither complete nor optimal. **IDDFS combines the best of both** — BFS's completeness/optimalty with DFS's linear space — by repeatedly running depth-limited DFS with a growing limit. The apparent waste of re-expanding shallow nodes is small because the last level dominates the count.

Likely exam questions

1. Differentiate BFS and DFS on strategy, completeness, optimality, time and space. **[10]**
2. What is iterative deepening search? Why does it combine the advantages of BFS and DFS? **[10]**
3. What is uniform-cost search? When is it optimal? **[5]**

MCQ traps

- **BFS uses a queue (FIFO); DFS uses a stack (LIFO).**
- BFS space is **$O(b^d)$** — its main weakness is memory, not time.
- **UCS** is optimal for any positive step costs; BFS only for equal costs.
- IDDFS space is **$O(bd)$** — linear.

3. Informed (heuristic) search — A* worked

Concept — in plain English first

Informed search uses a **heuristic** $h(n)$: an estimate of the cost from node n to the goal. This lets the search *prefer* promising directions instead of exploring blindly. Two main algorithms:

- **Greedy best-first:** expand the node with the smallest **$h(n)$** — nearest-looking to the goal. Fast, but **not optimal** and **not complete** (can be led astray, can loop).
- **A*:** expand the node with the smallest **$f(n) = g(n) + h(n)$** , where $g(n)$ = cost so far. It balances "how far I've come" with "how far I think is left". A* is **complete and optimal** provided h is admissible.

Key points — admissibility and consistency

- **Admissible heuristic:** never *overestimates* the true cost to the goal, $h(n) \leq h(n)$. *Admissibility guarantees A* returns an optimal** solution (tree search).
- **Consistent (monotone) heuristic:** $h(n) \leq \text{cost}(n, n') + h(n')$ for every neighbour. Consistency $\Rightarrow$ admissibility, and guarantees optimality for **graph** search too.
- A **more informed** (larger but still admissible) heuristic expands fewer nodes. $h(n)=0$ makes A* degenerate to UCS.

Worked A* example — do this by hand

Graph with start **A**, goal **G**. Edge costs (g) and heuristic estimates h to G:

Edges: A–B = 1, A–C = 4, B–C = 2, B–D = 5, C–D = 1, D–G = 3.

Heuristics: $h(A)=7$, $h(B)=6$, $h(C)=2$, $h(D)=3$, $h(G)=0$.

Compute $f = g + h$ as we expand, always expanding the frontier node with the lowest f .

Step	Expand	Frontier (node : $g + h = f$)	Chosen next
1	A ($g=0$)	B: $1+6=7$; C: $4+2=6$	C ($f=6$)
2	C ($g=4$)	B: $1+6=7$; D via C: $4+1+3 = 8$	B ($f=7$)
3	B ($g=1$)	D via B: $1+5+3=9$; D via C already 8; C already expanded. Best D = via C? Re-examine: D from B path $g=6$, from C path $g=5 \rightarrow$ keep $g(D)=5$, $f=5+3=8$	D ($f=8$)
4	D ($g=5$)	G: $g=5+3=8$, $h=0 \rightarrow f=8$	G — goal reached, cost 8

Optimal path: **A** $\rightarrow$ **C** $\rightarrow$ **D** $\rightarrow$ **G**, cost $4 + 1 + 3 = 8$. (Check the greedy path $A \rightarrow C \rightarrow D \rightarrow G$ is also what greedy-by- h would pick here, but in general greedy can miss the optimum; A^* is guaranteed because h is admissible — each $h \leq$ true remaining cost.)

Greedy vs A^* on the same graph: greedy expands by h alone ($A \rightarrow C$ because $h(C)=2$ is smallest, then $C \rightarrow D$ because $h(D)=3 < h(B)=6$, then $D \rightarrow G$) — here it happens to find the optimum, but had $A \rightarrow C$ been very expensive greedy would still have taken it because it ignores g . **A^* cannot be fooled that way.**

	Greedy best-first	A^*
Evaluation	$f = h(n)$	$f = g(n) + h(n)$
Complete	No	Yes
Optimal	No	Yes (h admissible)
Memory	Less	More (keeps all frontier)

Likely exam questions

1. What is a heuristic? Differentiate greedy best-first search and A^* search. [10]
2. Explain the A^* algorithm. Work through an example graph, computing $f = g + h$ at each step. [15]
3. What is an admissible heuristic? Why does admissibility guarantee A^* optimality? [10]
4. Differentiate admissible and consistent heuristics. [5]

MCQ traps

- **A^* : $f(n) = g(n) + h(n)$.** Greedy uses **$h(n)$** only; UCS uses **$g(n)$** only.
- A^* is optimal only if h is **admissible** (never overestimates).
- A more informed admissible heuristic expands **fewer** nodes.
- Consistency $\Rightarrow$ admissibility (not the reverse).

4. Adversarial search — minimax and alpha-beta

Concept — in plain English first

In a two-player, zero-sum game (chess, tic-tac-toe), one player (**MAX**) tries to maximise the score, the other (**MIN**) to minimise it. **Minimax** looks ahead down the game tree and assumes both play optimally: MAX picks the move leading to the highest value, MIN the lowest. **Alpha-beta pruning** is an optimisation of minimax that gives the *same answer* while skipping branches that cannot possibly

affect it.

Key points

- **Minimax value:** at a MAX node take the max of children; at a MIN node take the min. Leaves are evaluated by a **utility/evaluation function**.
- **Alpha (α):** best (highest) value MAX can guarantee so far. **Beta (β):** best (lowest) value MIN can guarantee so far. **Prune when $\alpha \geq \beta$** — the remaining children of that node cannot change the outcome.
- Alpha-beta gives the **identical** result to minimax. With perfect move ordering it examines $O(b^{(d/2)})$ leaves instead of $O(b^d)$ — effectively **doubling the searchable depth**.

Worked minimax + alpha-beta example

Game tree, MAX at the root, then MIN, then leaf values (left to right):



Plain minimax: MIN A = $\min(3, 5, 6) = 3$. MIN B = $\min(2, 9, 1) = 1$. MAX = $\max(3, 1) = 3$. Best move: go to A.

Alpha-beta on the same tree (left to right), $\alpha = -\infty$, $\beta = +\infty$ at root:

1. Enter A (MIN node), $\alpha = -\infty$, $\beta = +\infty$.
 - leaf 3 $\rightarrow \beta = 3$.
 - leaf 5 $\rightarrow 5 > \beta$? no, β stays 3.
 - leaf 6 $\rightarrow \beta$ stays 3. A returns 3.
2. Back at root (MAX): $\alpha = \max(-\infty, 3) = 3$.
3. Enter B (MIN node) with $\alpha = 3$, $\beta = +\infty$.
 - leaf 2 $\rightarrow \beta = 2$. **Now check $\alpha \geq \beta$: $3 \geq 2 \rightarrow$ PRUNE.** The remaining leaves (9 and 1) are **not examined** — MIN will pick at most 2 here, which is already worse for MAX than the 3 it has from A.
 - B returns 2 (its provisional value).
4. Root: $\max(3, 2) = 3$. Same answer, but **two leaves pruned**.

This — the tree, the minimax values, and the $\alpha \geq \beta$ prune with the skipped leaves named — is the 15-mark answer. Always state: alpha-beta yields the *same* value as minimax; ordering determines how much it prunes.

Likely exam questions

1. Explain the minimax algorithm for game playing with an example. [10]
2. What is alpha-beta pruning? Work an example showing which nodes are pruned. [15]
3. Why does alpha-beta produce the same result as minimax? What is the effect of move ordering? [10]

MCQ traps

- **MAX maximises, MIN minimises.** α = MAX's best-so-far, β = MIN's best-so-far.
- **Prune when $\alpha \geq \beta$.**
- Alpha-beta gives the **same** value as minimax (it is not an approximation).
- Best case examines **$O(b^{(d/2)})$** leaves — it does not change the *result*, only the *work*.

5. Constraint satisfaction problems (CSP)

Concept

A **CSP** is defined by **variables**, each with a **domain** of values, and **constraints** limiting which combinations are allowed. A solution assigns a value to every variable satisfying all constraints. Examples: map colouring, Sudoku, n-queens, timetabling. CSPs are solved by **backtracking search** plus pruning techniques.

Technique	Idea
Backtracking	Assign variables one at a time; backtrack when a constraint is violated
Forward checking	After each assignment, remove inconsistent values from neighbours' domains
Constraint propagation / arc consistency (AC-3)	Make every arc consistent: a value survives only if each constraint can still be satisfied
Heuristics	MRV (minimum remaining values — pick the most constrained variable), degree heuristic, least-constraining-value

Map colouring (colour a map so no two adjacent regions share a colour, given 3 colours) is the standard worked CSP.

Likely exam questions

1. What is a constraint satisfaction problem? Explain with the map-colouring example. **[10]**
2. Explain backtracking and forward checking for CSPs. **[5]**

MCQ traps

- **MRV** = pick the variable with the **fewest** legal values left.
- Forward checking prunes **neighbours'** domains after each assignment.

6. Knowledge and reasoning — building a knowledge base

Concept — in plain English first

A **knowledge-based agent** separates *what it knows* (the **knowledge base**, KB — a set of facts and rules expressed in a formal language) from *how it reasons* (the **inference engine** that derives new facts). This is the heart of the 2025 15-marker. The KB is built through **knowledge engineering** and queried by inference.

Key points — building a knowledge base (the 2025 Q19 answer)

Knowledge-engineering process (the steps to list):

1. **Identify the task** — what questions must the KB answer? (scope)
2. **Assemble the relevant knowledge** — knowledge acquisition from experts/sources.
3. **Decide on a vocabulary** — choose the predicates, functions, constants (the **ontology**).
4. **Encode general knowledge** — write the axioms/rules about the domain.
5. **Encode the specific problem instance** — the particular facts.
6. **Pose queries to the inference procedure** and get answers.
7. **Debug and refine** the knowledge base.

TELL and ASK: you **TELL** the KB new sentences and **ASK** it queries; the inference engine answers using what has been told.

Key challenges in building a knowledge base (the "challenges" the question demands):

Challenge	Why it is hard
Knowledge acquisition	Extracting expert knowledge is slow and error-prone (the "knowledge-acquisition bottleneck")
Representation choice	Which formalism (logic, frames, rules)? Trade-off expressiveness vs efficiency
Incompleteness & uncertainty	The world is not fully known; need default/probabilistic reasoning
Consistency	Avoiding contradictions as the KB grows
The frame problem	Representing what <i>stays the same</i> after an action, without listing everything
The qualification problem	Impossible to list every precondition of an action
Scalability & maintenance	Large KBs are hard to update and keep consistent
Common-sense knowledge	Vast, implicit, hard to formalise

Knowledge representation schemes (name and compare these)

Scheme	Idea	Strength / weakness
Logic (propositional/ predicate)	Facts and rules as logical sentences	Precise, sound inference; can be inefficient
Semantic networks	Graph of nodes (concepts) and labelled edges (relations, e.g. is-a, has-part); supports inheritance	Intuitive; ambiguous semantics
Frames	Objects with slots (attributes) and fillers; slots can have defaults; supports inheritance	Structured, OOP-like; the basis of many expert systems
Scripts	Stereotyped sequences of events (e.g. the "restaurant script")	Good for understanding narratives
Production rules	IF-THEN rules + working memory + inference engine	Modular, transparent; basis of expert systems

Properties a good representation should have: representational adequacy, inferential adequacy, inferential efficiency, acquisitional efficiency.

Likely exam questions

1. Explain the steps in building a knowledge base in AI and the key challenges involved. [15]
(*exactly the 2025 Q19*)
2. What is a knowledge-based agent? Explain TELL and ASK. [5]
3. Explain knowledge-representation schemes: logic, semantic networks, frames, scripts, rules. [10]
4. What is the frame problem? [5]

MCQ traps

- **Semantic networks** support **inheritance** via is-a links.
- **Frames** use **slots and fillers**; **scripts** represent stereotyped event sequences.
- **Production rules** = IF–THEN; used by expert systems.

7. Logic and reasoning — unification and resolution

Concept — in plain English first

Logic gives a KB a precise language and *sound* inference. **Propositional logic** deals with whole statements (P, Q) combined by $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$. **Predicate (first-order) logic** adds **objects**, **predicates**, **functions** and **quantifiers** ($\forall$ "for all", $\exists$ "there exists"), so it can say "every man is mortal" rather than needing one proposition per man. **Resolution** is a single, complete inference rule that, with **refutation** (proof by contradiction), can prove any entailed sentence.

Key points — propositional and predicate logic

	Propositional	Predicate (FOL)
Units	Atomic propositions P, Q	Predicates over objects: Man(x), Loves(x,y)
Quantifiers	none	$\forall$ (universal), $\exists$ (existential)
Expressiveness	limited	can quantify over objects

Inference rules: Modus Ponens (from P and $P \rightarrow Q$ infer Q), Modus Tollens, And-elimination, resolution. **Validity/satisfiability:** *valid* = true in all models (tautology); *satisfiable* = true in some model; *unsatisfiable* = true in none.

7.1 Unification

Unification finds a substitution that makes two predicate expressions identical. It is the engine of predicate-logic inference.

- Unify `Knows(John, x)` and `Knows(John, Jane)` $\rightarrow \{x/Jane\}$.
- Unify `Knows(John, x)` and `Knows(y, Mother(y))` $\rightarrow \{y/John, x/Mother(John)\}$.
- Unify `Knows(John, x)` and `Knows(x, Jane)` $\rightarrow$ **fails** (x cannot be both John and Jane) — unless standardised apart.

The **most general unifier (MGU)** is the least-committing substitution that unifies them.

7.2 Resolution and refutation — worked

Resolution rule (propositional): from clauses $(A \vee B)$ and $(\neg B \vee C)$ infer $(A \vee C)$ — cancel the complementary literals B and $\neg B$.

Proof by refutation: to prove $KB \models \alpha$, add $\neg\alpha$ to the KB, convert everything to **conjunctive normal form (CNF)** clauses, and apply resolution repeatedly. If you derive the **empty clause** (a contradiction), then α is proved.

Worked example — "Did Marcus hate Caesar?" style / the classic syllogism

Prove **Mortal(Socrates)** from:

- $\forall x (\text{Man}(x) \rightarrow \text{Mortal}(x))$
- $\text{Man}(\text{Socrates})$

Step 1 — to clauses (CNF).

- (1) becomes $\neg \text{Man}(x) \vee \text{Mortal}(x)$
- (2) is $\text{Man}(\text{Socrates})$

Step 2 — negate the goal. Add $\neg \text{Mortal}(\text{Socrates})$.

Step 3 — resolve.

- Resolve $\neg \text{Man}(x) \vee \text{Mortal}(x)$ with $\text{Man}(\text{Socrates})$, unifier $\{x/\text{Socrates}\} \rightarrow \text{Mortal}(\text{Socrates})$.
- Resolve $\text{Mortal}(\text{Socrates})$ with $\neg \text{Mortal}(\text{Socrates}) \rightarrow$ **empty clause** $\square$.

Contradiction derived $\Rightarrow$ **Mortal(Socrates) is proved. ■**

Steps to convert to CNF (worth listing): eliminate $\rightarrow$ and $\leftrightarrow$; move $\neg$ inwards (De Morgan); standardise variables apart; Skolemise (remove $\exists$); drop $\forall$; distribute $\vee$ over $\wedge$; rename.

7.3 Forward vs backward chaining

	Forward chaining	Backward chaining
Direction	Data $\rightarrow$ conclusions (start from facts, fire rules)	Goal $\rightarrow$ facts (start from the query, find rules that conclude it)
Driven by	Data-driven	Goal-driven
Good for	Deriving everything that follows (monitoring)	Answering a specific query (diagnosis)
Used by	Production systems (e.g. Rete)	PROLOG , expert-system diagnosis

Likely exam questions

- Differentiate propositional and predicate logic. [5]
- What is unification? Give the MGU for two example expressions. [5]
- Explain resolution refutation. Prove $\text{Mortal}(\text{Socrates})$ from the man/mortal axioms. [15]
- Differentiate forward and backward chaining. [10]

MCQ traps

- Resolution refutation** derives the **empty clause** to prove entailment.
- Resolution needs the KB in **CNF (clause form)**.
- PROLOG uses backward chaining**; forward chaining is data-driven.
- $\forall$ = universal, $\exists$ = existential; **Skolemisation** removes $\exists$.

8. Reasoning under uncertainty — Bayes worked

Concept — in plain English first

Real knowledge is uncertain. **Probability** lets an agent believe things to a degree and update those beliefs with evidence. **Bayes' theorem** is the update rule: it turns "probability of evidence given a cause" into "probability of the cause given the evidence" — exactly what diagnosis needs.

Key points — Bayes' theorem

$$P(H \mid E) = P(E \mid H) \cdot P(H) / P(E)$$

where $P(H)$ = prior, $P(E|H)$ = likelihood, $P(H|E)$ = posterior, and
 $P(E) = P(E|H) \cdot P(H) + P(E|\neg H) \cdot P(\neg H)$ (total probability).

Worked Bayes example — the classic medical test

A disease affects **1%** of a population. A test is **99%** sensitive ($P(+|disease)=0.99$) and has a **5%** false-positive rate ($P(+|no\ disease)=0.05$). A person tests positive. What is $P(disease \mid +)$?

- Prior: $P(D) = 0.01$, $P(\neg D) = 0.99$.
- $P(+) = P(+|D) \cdot P(D) + P(+|\neg D) \cdot P(\neg D) = 0.99 \times 0.01 + 0.05 \times 0.99 = 0.0099 + 0.0495 = \mathbf{0.0594}$.
- $P(D \mid +) = P(+|D) \cdot P(D) / P(+) = 0.0099 / 0.0594 = \mathbf{0.1667 \approx 16.7\%}$.

The counter-intuitive punchline (state it): despite a very accurate test, a positive result means only ~17% chance of disease, because the disease is rare — the false positives from the healthy 99% swamp the true positives. This "base-rate" effect is the whole point of the example.

Bayesian networks

A **Bayesian network** is a **directed acyclic graph** where nodes are random variables and edges are direct probabilistic dependencies; each node has a **conditional probability table (CPT)** given its parents. It compactly encodes the joint distribution as $P(x_1, \dots, x_n) = \prod P(x_i \mid \text{parents}(x_i))$, exploiting conditional independence. Used for diagnosis, prediction, and explanation.

Fuzzy logic

Where probability handles *uncertainty*, **fuzzy logic** handles *vagueness*: truth values are a **degree in [0,1]**, not just 0/1. A temperature can be "0.7 hot and 0.3 warm". Components: **fuzzification** (crisp → fuzzy membership), a **rule base** (IF temperature is hot THEN fan is fast), **inference**, and **defuzzification** (fuzzy → crisp output). Used in control systems (washing machines, air conditioners).

	Probability	Fuzzy logic
Handles	Uncertainty (likelihood of an event)	Vagueness (degree of membership)
Value	Chance in [0,1]	Membership degree in [0,1]
Example	"70% chance of rain"	"the day is 0.7 rainy"

Likely exam questions

1. State Bayes' theorem. A disease test problem — compute the posterior. [10]
2. What is a Bayesian network? How does it represent a joint distribution? [10]
3. What is fuzzy logic? Differentiate it from probability. Explain fuzzification and defuzzification. [10]

MCQ traps

- Bayes: $P(H|E) = P(E|H)P(H)/P(E)$.
- A Bayesian network is a **DAG** with **CPTs**; it exploits **conditional independence**.
- Fuzzy logic uses **degrees of membership**; it is about **vagueness**, not probability.

9. Learning from observations — decision trees (ID3) worked

Concept — in plain English first

Inductive learning generalises from examples to a rule. A **decision tree** is the classic inductive learner: it asks a sequence of attribute questions, each narrowing down the class, ending in a leaf that predicts the label. **ID3** builds the tree greedily by always splitting on the attribute that gives the greatest **information gain** — the attribute that most reduces uncertainty (entropy).

Key points — entropy and information gain

Entropy of a set S with class proportions p_+ , p_- :

$H(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$ (0 = pure, 1 = 50/50 for two classes).

Information Gain of splitting S on attribute A :

$\text{Gain}(S, A) = H(S) - \sum_v (|S_v|/|S|) \cdot H(S_v)$ (parent entropy minus weighted child entropy).

ID3: at each node pick the attribute with the **highest information gain**; recurse.

Worked ID3 example — the "Play Tennis" style dataset (first split)

14 examples, target = PlayTennis (Yes/No). Overall: **9 Yes, 5 No**.

Root entropy: $H(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$
 $= -(0.643)(-0.637) - (0.357)(-1.485) = 0.410 + 0.530 = \mathbf{0.940}$.

Gain for the attribute Outlook (values Sunny, Overcast, Rain):

Value	count	Yes	No	Entropy of subset
Sunny	5	2	3	$-(2/5)\log_2(2/5) - (3/5)\log_2(3/5) = 0.971$
Overcast	4	4	0	0 (pure)
Rain	5	3	2	0.971

Weighted child entropy = $(5/14)(0.971) + (4/14)(0) + (5/14)(0.971) = 0.347 + 0 + 0.347 = \mathbf{0.694}$.

Gain(S, Outlook) = 0.940 - 0.694 = 0.247.

Comparable computations give (standard dataset values): Gain(Humidity)=0.151, Gain(Wind)=0.048, Gain(Temperature)=0.029. **Outlook has the highest gain → it is the root split.** The Overcast branch is pure (all Yes) → a leaf; recurse on the Sunny and Rain subsets. This computation — root entropy, per-attribute weighted entropy, gain, pick the max — is the full worked answer.

Extensions to name: C4.5 uses **gain ratio** (to correct ID3's bias toward many-valued attributes) and handles continuous attributes and missing values; **CART** uses the **Gini index**. Trees over-fit, so they are **pruned**.

Inductive vs explanation-based learning (the syllabus names both)

	Inductive learning	Explanation-based learning (EBL)
Input	Many labelled examples	One example + a domain theory
Learns by	Generalising over examples (data-driven)	Explaining <i>why</i> one example is an instance, then generalising the explanation (knowledge-driven)
Needs prior knowledge	Little	A strong domain theory
Output	A hypothesis fitting the data	An operational rule / a proof-based generalisation
Risk	Over/under-fitting	Only as good as the domain theory

EBL turns knowledge you already have into a *faster* form (speed-up learning): it doesn't learn new facts, it re-expresses them usefully. Inductive learning genuinely acquires new generalisations from data.

Likely exam questions

1. What is inductive learning? Explain decision-tree learning (ID3) with entropy and information gain, working a first split. **[15]**
2. Differentiate inductive learning and explanation-based learning. **[10]**
3. What is entropy and information gain? Why does ID3 use them? **[5]**

MCQ traps

- **ID3 splits on maximum information gain**; entropy 0 = pure node.
- **C4.5 = gain ratio**; **CART = Gini index**.
- **EBL** needs a **domain theory** and learns from **one** example (speed-up learning).
- Entropy of a 50/50 two-class set = 1.

10. Pattern recognition and expert systems

Concept — pattern recognition

Pattern recognition assigns an input (image, sound, signal) to a category. The syllabus names four facets:

Term	Meaning
Structured description	Represent a pattern by its parts and their relations (e.g. a face = eyes + nose + mouth arranged so) — <i>syntactic/structural PR</i>
Symbolic description	Represent a pattern by symbolic features/attributes rather than raw pixels

Object identification	Recognise/label objects in a scene (computer vision)
Speech recognition	Map an acoustic signal to words (HMMs classically, deep nets now)

PR approaches: *statistical* (feature vectors + a classifier, e.g. Bayes, k-NN), *structural / syntactic* (grammars of sub-patterns), *template matching*, *neural networks*. Pipeline: **sensing → preprocessing → feature extraction → classification → post-processing.**

Concept — expert systems

An **expert system** emulates a human expert in a narrow domain. Architecture:

Component	Role
Knowledge base	Domain facts + IF–THEN rules
Inference engine	Applies rules (forward/backward chaining) to derive conclusions
Working memory	Current facts about the case
Explanation facility	Explains <i>why</i> / <i>how</i> a conclusion was reached
User interface	Dialogue with the user
Knowledge-acquisition module	Helps the knowledge engineer add knowledge

Classic examples: **MYCIN** (diagnosing infections), **DENDRAL** (chemistry). Strengths: consistency, availability, explanation. Weaknesses: brittle outside their domain, knowledge-acquisition bottleneck, no common sense.

Likely exam questions

1. What is pattern recognition? Explain structured vs symbolic description and object identification. **[10]**
2. Explain the architecture of an expert system with a diagram. **[10]**
3. What are the components of an expert system? Give two examples. **[5]**

MCQ traps

- The **inference engine** applies rules; the **knowledge base** stores them.
- **MYCIN** = medical diagnosis expert system; **DENDRAL** = chemistry.
- Expert systems lack **common-sense** knowledge.

11. AI programming languages — LISP and PROLOG

Concept — in plain English first

The syllabus names two AI languages. **LISP** is a **functional** language built on lists and recursion — code and data share the same list form, which makes it natural for symbolic manipulation.

PROLOG is a **logic** language: you state facts and rules, and the system answers queries by automatic backward-chaining search with unification. LISP says *how* to compute; PROLOG says *what* is

true and lets the engine find the answer.

LISP (LISt Processing)

- Everything is an **S-expression** in prefix (Cambridge Polish) notation: `(operator operand ...)`.
- `(+ 2 3) → 5`. `(* (+ 1 2) 4) → 12`.
- Core list functions: **car** (first element), **cdr** (rest), **cons** (prepend), **list**, **append**, **null**, **atom**.
- Functions via **defun**; recursion is the main control structure; **cond/if** for branching.

Example — factorial in LISP:

```
(defun factorial (n)
  (if (<= n 1)
      1
      (* n (factorial (- n 1)))))
(factorial 5) ; => 120
```

Example — list length:

```
(defun len (lst)
  (if (null lst) 0 (+ 1 (len (cdr lst)))))
```

PROLOG (PROgramming in LOGic)

- Program = **facts + rules**; you run it by posing **queries (goals)**.
- Fact: `parent(tom, bob)`. Rule: `grandparent(X, Z) :- parent(X, Y), parent(Y, Z)`.
(`:-` reads "if"; `,` is AND; uppercase = variable.)
- Execution = **backward chaining + unification + backtracking**.

Example — family relations:

```
parent(tom, bob).
parent(bob, ann).
parent(bob, pat).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

?- grandparent(tom, ann).    % => true
?- grandparent(tom, W).     % => W = ann ; W = pat
```

Example — factorial:

```
factorial(0, 1).
factorial(N, F) :- N > 0, N1 is N-1, factorial(N1, F1), F is N*F1.
?- factorial(5, X).    % => X = 120
```

LISP vs PROLOG — the comparison table

	LISP	PROLOG
Paradigm	Functional	Logic (declarative)
Basic unit	S-expression / list	Fact and rule (Horn clauses)

Computation	Function evaluation, recursion	Unification + backward chaining + backtracking
You specify	How to compute	What is true
Control	Explicit (recursion, cond)	Implicit (the resolution engine)
Typical use	Symbolic AI, early AI systems	Expert systems, NLP, theorem proving

Likely exam questions

1. Differentiate LISP and PROLOG. **[10]**
2. Write a LISP program for factorial / list length. **[5]**
3. Write PROLOG facts and rules for family relationships and a query. **[5]**
4. Explain how PROLOG evaluates a query (unification, backtracking). **[10]**

MCQ traps

- **LISP** = functional, list-based, prefix notation; **PROLOG** = logic, facts+rules, backward chaining.
- `car` = first, `cdr` = rest, `cons` = construct.
- In PROLOG `:-` means "if", `,` means AND, uppercase are variables.
- PROLOG uses **backtracking** to find alternative solutions.

PART B — MACHINE LEARNING (Unit 7)

12. Machine-learning fundamentals

Concept — in plain English first

Machine learning builds programs that **improve at a task with experience** instead of being explicitly programmed. Formally (Mitchell): a program learns from experience **E** with respect to task **T** and performance measure **P** if its performance on T (measured by P) improves with E.

Key points — the three paradigms

Paradigm	Data	Goal	Examples
Supervised	Labelled (input, output)	Learn input→output mapping	Classification (spam, digit), regression (price) — k-NN, decision trees, SVM, linear regression
Unsupervised	Unlabelled inputs only	Find structure	Clustering (K-means), dimensionality reduction (PCA), association rules
Reinforcement	Reward/penalty signal from an environment	Learn a policy that maximises cumulative reward	Q-learning, game playing, robotics

Classification vs regression: classification predicts a **discrete class**; regression predicts a **continuous value**.

12.1 Knowledge-level vs symbol-level learning (the 2025 Q14 answer)

This distinction (from Dietterich) is *named explicitly* in the syllabus and was a 10-marker in 2025.

	Knowledge-level learning	Symbol-level learning
What changes	The agent acquires new knowledge — it can now answer questions/derive conclusions it could not before	The agent reorganises/represents existing knowledge more efficiently — it learns nothing new logically, just computes faster
Effect on competence	Increases what the agent knows (extends deductive closure)	No new deductive consequences — same conclusions, reached faster
Analogy	Learning a new fact or rule	Learning a shortcut / caching / re-indexing
Example	Inductive learning from examples (genuinely new generalisation)	Explanation-based learning (speed-up learning), macro-operators, caching

One-line answer: *knowledge-level* learning adds to what the system can deduce; *symbol-level* learning only makes existing deductions cheaper. EBL is the canonical symbol-level learner; induction from data is the canonical knowledge-level learner.

Likely exam questions

1. What is machine learning? Differentiate supervised, unsupervised and reinforcement learning. [10]
2. Differentiate knowledge-level and symbol-level learning with examples. [10] (2025 Q14)
3. Differentiate classification and regression. [5]

MCQ traps

- **Supervised = labelled; unsupervised = unlabelled; reinforcement = reward-driven.**
- Clustering is **unsupervised**; classification is **supervised**.
- **Symbol-level** learning yields **no new deductive consequences** (EBL); knowledge-level does.

13. Supervised learning — k-Nearest Neighbours (worked)

Concept — in plain English first

k-NN is the simplest supervised classifier: to classify a new point, look at its **k nearest labelled neighbours** and take a **majority vote** of their classes (for regression, average their values). It is a **lazy** learner — it does no training; it just stores the data and defers all work to query time.

Key points

- **Distance metrics:**
- **Euclidean:** $\sqrt{\sum (x_i - y_i)^2}$ (straight-line, most common)
- **Manhattan (city-block):** $\sum |x_i - y_i|$
- **Minkowski:** $(\sum |x_i - y_i|^p)^{1/p}$ — generalises both ($p=2$ Euclidean, $p=1$ Manhattan)

- **Hamming**: number of differing positions (categorical)
- **Choosing k**: small k → sensitive to noise (over-fits); large k → over-smooths (under-fits). Use an **odd** k for two classes to avoid ties; tune by cross-validation. Rule of thumb $k \approx \sqrt{n}$.
- **Feature scaling is essential** — features on larger numeric ranges dominate the distance, so normalise/standardise first.

Worked k-NN classification

Training data (2 features, class in {A, B}):

Point	x1	x2	Class
P1	1	1	A
P2	2	2	A
P3	4	4	B
P4	5	5	B
P5	1	2	A

Classify the query **Q = (3, 3)** with **k = 3**. Euclidean distances:

Point	distance to (3,3)	Class
P1 (1,1)	$\sqrt{(4+4)}=2.83$	A
P2 (2,2)	$\sqrt{(1+1)}=1.41$	A
P3 (4,4)	$\sqrt{(1+1)}=1.41$	B
P4 (5,5)	$\sqrt{(4+4)}=2.83$	B
P5 (1,2)	$\sqrt{(4+1)}=2.24$	A

Three nearest (smallest distance): **P2 (1.41, A), P3 (1.41, B), P5 (2.24, A)**.

Vote: A appears twice, B once → **predict class A**. (With k=1 there is a tie between P2 and P3; tie-breaking by nearest/first gives A or B — a good point to mention.)

Factors influencing k-NN performance (the 2025 Q15 answer)

Factor	Effect
Choice of k	Too small → noisy/over-fit; too large → over-smoothed/under-fit
Distance metric	Must suit the data (Euclidean vs Manhattan vs Hamming)
Feature scaling	Unscaled features distort distances — normalise
Irrelevant features / dimensionality	The curse of dimensionality (below)
Data size	Query cost grows with n (lazy, stores all data)
Class imbalance	Majority class can dominate the vote; use weighting

Curse of dimensionality: as the number of features grows, points become almost equidistant, "near" loses meaning, data becomes sparse, and k-NN degrades badly. Mitigate with feature selection / dimensionality reduction.

Pros/cons: simple, no training, naturally multi-class, adapts as data is added; but slow at query time ($O(n)$ per query), memory-heavy, sensitive to scaling and irrelevant features.

Likely exam questions

1. Explain the k-NN algorithm. What factors influence its performance? **[10]** (2025 Q15)
2. Work a k-NN classification for a given dataset and query with $k=3$. **[10]**
3. What is the curse of dimensionality? How does it affect k-NN? **[5]**
4. Compare Euclidean and Manhattan distance. **[5]**

MCQ traps

- k-NN is a **lazy** learner — **no training phase**; all work at query time.
- **Odd k** avoids ties for two classes.
- **Feature scaling matters** for distance-based methods.
- Small $k \rightarrow$ over-fit; large $k \rightarrow$ under-fit.

14. Unsupervised learning — K-means (worked)

Concept — in plain English first

Clustering groups unlabelled points so that points in a group are similar and points in different groups are dissimilar. **K-means** is the standard: pick k cluster centres, assign each point to its nearest centre, move each centre to the mean of its assigned points, and repeat until nothing changes. It minimises the total **within-cluster sum of squares (WCSS)**.

Key points — the algorithm

1. Choose k ; initialise k centroids (randomly or k-means++).
2. **Assignment step:** assign each point to the **nearest centroid** (Euclidean).
3. **Update step:** recompute each centroid as the **mean** of its assigned points.
4. Repeat 2–3 until assignments stop changing (convergence).

Properties: converges to a **local** optimum (result depends on initialisation — run several times); must choose k in advance (**elbow method** on WCSS, or silhouette score); sensitive to outliers and scaling; assumes roughly spherical, equal-size clusters. Complexity $O(n \cdot k \cdot i \cdot d)$.

Worked K-means iteration

1-D points: {2, 4, 10, 12, 3, 20, 30, 11, 25}, $k = 2$. Initial centroids $c1 = 2$, $c2 = 4$.

Iteration 1 — assign (nearest centroid):

- Cluster 1 (near 2): {2, 3} $\rightarrow$ closer to 2 than 4.
- Cluster 2 (near 4): {4, 10, 12, 20, 30, 11, 25}.

Update: $c1 = \text{mean}\{2,3\} = 2.5$; $c2 = \text{mean}\{4,10,12,20,30,11,25\} = 112/7 = 16$.

Iteration 2 — reassign to {2.5, 16}:

- Cluster 1 (near 2.5): {2, 3, 4} (4 is 1.5 from 2.5 vs 12 from 16).
- Cluster 2 (near 16): {10, 12, 20, 30, 11, 25}.

Update: $c1 = \text{mean}\{2,3,4\} = 3$; $c2 = \text{mean}\{10,12,20,30,11,25\} = 108/6 = 18$.

Iteration 3 — reassign to {3, 18}:

- Cluster 1: {2, 3, 4, 10} (10 is 7 from 3 vs 8 from 18).
- Cluster 2: {12, 20, 30, 11, 25}.

Update: $c1 = \text{mean}\{2,3,4,10\} = 4.75$; $c2 = \text{mean}\{12,20,30,11,25\} = 98/5 = 19.6$.

Continue until assignments stabilise. This step-by-step assign/update table is the 15-mark worked answer — show at least two full iterations and state "repeat until no point changes cluster".

Kernel K-means and hierarchical clustering

- **Kernel K-means:** map points into a higher-dimensional feature space (via a **kernel function**, e.g. RBF) and run K-means there, so it can find **non-linearly separable** / **non-spherical** clusters that plain K-means cannot. Uses the kernel trick — distances computed via the kernel without explicit mapping.
- **Hierarchical clustering:** builds a tree (**dendrogram**) of nested clusters.
- **Agglomerative (bottom-up):** start with each point as its own cluster, repeatedly merge the two closest. Linkage: single (min), complete (max), average.
- **Divisive (top-down):** start with one cluster, repeatedly split.
- No need to pre-choose k (cut the dendrogram at the desired level), but $O(n^2)$ or worse.

	K-means	Hierarchical
Need k in advance	Yes	No (cut dendrogram)
Output	Flat partition	Dendrogram (nested)
Complexity	$O(nki)$	$O(n^2 \log n)$ or $O(n^3)$
Re-run stability	Depends on init	Deterministic (given linkage)

Likely exam questions

1. Explain the K-means algorithm. Work through an example with $k=2$ for at least two iterations. [15]
2. What are the limitations of K-means? What is kernel K-means? [10]
3. Differentiate K-means and hierarchical clustering. [10]
4. What is agglomerative clustering? Explain single vs complete linkage. [5]

MCQ traps

- K-means minimises **within-cluster sum of squares**; converges to a **local** optimum.
- k must be **chosen in advance** (elbow method); result depends on **initialisation**.
- **Kernel K-means** handles **non-linearly separable** clusters.
- Agglomerative = **bottom-up**; divisive = **top-down**.
- Clustering is **unsupervised**.

15. Statistical learning — ensemble methods

Concept — in plain English first

An **ensemble** combines many "weak" models into one strong model — a committee outperforms any single member. The two dominant strategies attack different problems: **bagging** reduces **variance** (by averaging independent models), **boosting** reduces **bias** (by focusing new models on earlier mistakes).

Key points

Method	Idea	Reduces	Trains models
Bagging (Bootstrap Aggregating)	Train many models on bootstrap samples (random samples with replacement); average/vote their outputs	Variance	in parallel (independent)
Boosting	Train models sequentially ; each new model focuses on the examples the previous ones got wrong; weighted vote	Bias	sequentially (dependent)
Random Forest	Bagging of decision trees + random feature subset at each split (extra de-correlation)	Variance	parallel

- **AdaBoost (Adaptive Boosting)**: start with equal example weights; train a weak learner; **increase the weights of misclassified examples**; give each learner a vote weight based on its accuracy; combine as a weighted majority. Sensitive to noise/outliers (it chases hard examples).
- **Random Forest** = many decision trees, each on a bootstrap sample and choosing splits from a random subset of features; predict by majority vote. Robust, little tuning, gives feature-importance; the workhorse ensemble.
- **Gradient Boosting** (name it): boosting that fits each new tree to the **residual errors** of the current ensemble (XGBoost, LightGBM).

	Bagging	Boosting
Training	Parallel, independent	Sequential, dependent
Sample	Bootstrap (equal weight)	Reweighted toward errors
Combines by	Simple average/vote	Weighted vote
Attacks	Variance (over-fitting)	Bias (under-fitting)
Over-fitting risk	Low	Higher (noise sensitive)
Example	Random Forest	AdaBoost, Gradient Boosting

Bias–variance trade-off

- **Bias** = error from wrong assumptions (too simple a model → **under-fitting**).
- **Variance** = error from sensitivity to the training set (too complex → **over-fitting**).
- Total error = bias² + variance + irreducible noise. You cannot minimise both freely; the goal is the sweet spot. **Bagging lowers variance; boosting lowers bias.**

Likely exam questions

1. What are ensemble methods? Differentiate bagging and boosting. [10]
2. Explain AdaBoost. [10]

3. What is a random forest? Why does it usually beat a single decision tree? [10]
4. Explain the bias–variance trade-off. [10]

MCQ traps

- **Bagging** → parallel, reduces variance; **boosting** → sequential, reduces bias.
- **Random forest** = bagging of trees + random feature subsets.
- **AdaBoost** re-weights misclassified examples upward.
- High bias = under-fit; high variance = over-fit.

16. Model evaluation — confusion matrix, precision/recall/F1

Concept — in plain English first

To judge a classifier you need more than "accuracy" — especially with imbalanced classes. The **confusion matrix** tabulates predictions vs truth, and from it come precision, recall, and F1.

Key points — the confusion matrix (binary)

	Predicted +	Predicted –
Actual +	TP (true positive)	FN (false negative)
Actual –	FP (false positive)	TN (true negative)

Metrics:

Metric	Formula	Meaning
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall correctness (misleading if imbalanced)
Precision	$TP/(TP+FP)$	Of those <i>predicted</i> positive, how many really are
Recall (Sensitivity, TPR)	$TP/(TP+FN)$	Of the <i>actual</i> positives, how many were caught
Specificity (TNR)	$TN/(TN+FP)$	Of actual negatives, how many caught
F1-score	$2 \cdot (P \cdot R) / (P + R)$	Harmonic mean of precision & recall (balances both)

Precision vs recall trade-off: raising the decision threshold usually raises precision but lowers recall, and vice versa. Use **precision** when false positives are costly (spam filter), **recall** when false negatives are costly (disease/fraud detection). **F1** balances the two.

Worked example

A model gives: **TP = 40, FP = 10, FN = 20, TN = 30.**

- Accuracy = $(40+30)/100 = 0.70$
- Precision = $40/(40+10) = 40/50 = 0.80$
- Recall = $40/(40+20) = 40/60 = 0.667$

- $F1 = 2 \cdot (0.80 \cdot 0.667) / (0.80 + 0.667) = 2 \cdot (0.533) / (1.467) = \mathbf{0.727}$

Overfitting, underfitting and cross-validation

- **Overfitting:** model fits training noise → great on training data, poor on new data (high variance). Symptoms: large train–test gap. Cures: more data, regularisation, pruning, simpler model, dropout, early stopping.
- **Underfitting:** model too simple → poor on both (high bias). Cure: more complex model, more features.
- **Cross-validation:** split data to estimate generalisation honestly. **k-fold CV:** split into k folds, train on k–1, test on the held-out fold, rotate, average. **LOOCV** = k = n. Used for model selection and hyper-parameter tuning (choosing k in k-NN, tree depth). The **train/validation/test** split keeps the final test set untouched until the end.

Likely exam questions

1. What is a confusion matrix? Define precision, recall and F1, and compute them for given TP/FP/FN/TN. **[10]**
2. Differentiate overfitting and underfitting. How do you detect and prevent overfitting? **[10]**
3. What is cross-validation? Explain k-fold cross-validation. **[5]**
4. When would you prefer precision over recall, and vice versa? **[5]**

MCQ traps

- **Precision = $TP/(TP+FP)$; Recall = $TP/(TP+FN)$.** (Denominators constantly swapped.)
- **F1 is the harmonic mean** of precision and recall.
- Accuracy is **misleading** on imbalanced data.
- Overfitting = **high variance** (great train, poor test); underfitting = **high bias**.
- **k-fold CV** trains on k–1 folds, tests on the remaining one, k times.

Exam armour — one-page recall for this unit

Prompt	Instant answer
A* evaluation function	$f(n) = g(n) + h(n)$; admissible $h \Rightarrow$ optimal
Greedy vs UCS	greedy uses h only; UCS uses g only
Alpha-beta prune condition	$\alpha \geq \beta$
Bayes	$P(H E) = P(E H)P(H)/P(E)$
Resolution proof	negate goal, to CNF, derive empty clause
PROLOG inference	backward chaining + unification + backtracking
ID3 split criterion	maximum information gain (entropy reduction)
Knowledge- vs symbol-level	new knowledge vs faster same knowledge (EBL)

k-NN	lazy, majority vote of k nearest; scale features; odd k
K-means steps	assign to nearest centroid, update to mean, repeat
Kernel K-means	handles non-spherical clusters via kernel trick
Bagging vs boosting	parallel/variance vs sequential/bias
Random forest	bagged trees + random feature subsets
Precision / Recall	$TP/(TP+FP)$ / $TP/(TP+FN)$
F1	harmonic mean of P and R
Overfit vs underfit	high variance vs high bias
Supervised/unsup/RL	labelled / unlabelled / reward

Closing note. AI and ML never appear in Paper-II Section A — they are your **10- and 15-mark** territory. The three things to have automatic from a blank page: **A* on a small graph**, the **ID3 first-split** entropy/gain computation, and a **K-means iteration**. Add the two direct 2025 repeats — **knowledge-level vs symbol-level** and **k-NN factors** — and you have banked most of the unit's marks.